Floating-Point Numbers

Algorithms, Programming, and Data Representation · Module 92313 · Chapter 4

From Fundamentals to AI Apocalypse · personal study guide

Contents

Start from memory 2

How to use this guide 2

How the chapter fits together 3

4.1 The floating-point model 4

4.2 Binary fractions 5

4.3 Floating-point arithmetic and error 7

4.4 IEEE 754 binary formats 9

4.5 Zero, subnormal numbers, infinity, and NaN 11

Formula map 13

Derive it cold 13

Explain it 14

Source check 14

Mixed practice 15

Exam-style problems 16

Self-check 17

Answers 18

Floating-point formats encode a wide range of magnitudes with a sign, a significand, and an exponent. This guide builds from binary fractions to IEEE single and double precision, then shows why rounding and special values matter.

Start from memory

Try each question before reading.

Mark the answer sure, unsure, or guessed.

1. In a floating-point number, what does the exponent control?
2. Why can the decimal fraction 0.1 have no finite binary expansion?
3. What must happen before two floating-point values can be added?
4. How many stored bits are in IEEE single precision?
5. What is the purpose of the hidden leading bit in a normalised binary value?

Answers: see "Start from memory" under Answers at the end.

How to use this guide

Study one topic, hide it, and reconstruct the rules and examples from memory.

Attempt mixed practice before opening its worked answers.

1. **Day 0:** Try the cold start and study §4.1. Explain range and precision in your own words.
2. **Day 1:** Rebuild the binary fraction place values in §4.2; solve practice 1–5.
3. **Day 3:** Work through multiplication, addition, rounding, and exponent limits in §4.3; solve practice 6–12.
4. **About day 7:** Reconstruct the single-precision fields and conversions in §4.4; solve practice 13–17.
5. **About day 14:** Recall the special-value rules in §4.5; complete both exam-style problems without notes.
6. **Before the exam:** Tick only skills you can demonstrate cold; revisit the sections named in missed answers.

When stuck: state the base, format, sign, and exponent convention.

Write every shift and rounding step, then check the magnitude of the result.

How the chapter fits together

Floating-point representation trades a fixed range of equally spaced values for a range whose spacing changes with magnitude.

Binary fractions give the significand its value.

Arithmetic aligns or combines significands and then rounds to the available precision.

Represent · choose a scale and precision

- **Sign, significand, exponent** §4.1

  value = sign × significand × $\text{base}^{\text{exponent}}$

  Rests on: scientific notation separates a number's digits from its scale.

- **Binary fractions** §4.2

  $0.b_1b_2\ldots = \Sigma\ b_i 2^{-i}$

  Rests on: each fractional bit has a negative power-of-two place value.

Compute · preserve approximate value

- **Floating-point arithmetic** §4.3

  multiply: multiply significands; add exponents

  Rests on: exponent laws and normalisation after an operation.

- **Rounding and limits** §4.3

  finite precision → rounding; finite range → overflow or underflow

  Rests on: a format can store only finitely many significand and exponent patterns.

Decode · read the stored fields

- **IEEE formats** §4.4

  sign + biased exponent + fraction

  Rests on: a standard layout makes bit patterns portable between systems.

- **Special values** §4.5

  zero · subnormal · infinity · NaN

  Rests on: reserved exponent patterns encode values outside the normalised range.

Question	Useful method
What value does a pattern represent?	Read sign, exponent, fraction, and format rules.
Can a decimal fraction be exact in binary?	Expand it in powers of 2 or test whether the expansion terminates.
How do I add two values?	Restore hidden bits, align exponents, add signed significands, normalise, and round.
Why did a computation change a value?	Check finite precision, exponent range, and special-value rules.

4.1 The floating-point model

A fixed number of bits cannot represent every real number.

Floating point uses a sign, a significand, and an exponent to cover very large and very small magnitudes.

Range and precision

For base β, a normalised number has the form sign × significand × β^{exponent}.

The exponent controls scale and therefore the approximate range.

The number of significand bits controls precision, or how finely nearby values can be distinguished.

Floating-point values are finite in number, and their spacing changes with magnitude.

Scientific notation model

$$x = s \times M \times \beta^{E}$$

s is the sign, M is the significand, β is the base or radix, and E is the exponent. In binary floating point, β=2.

Worked example · normalise a decimal value

Write 343000×10^{-4} in normalised decimal form with $1 \leq M < 10$.

1. Move the decimal point in 343000 five places left to make 3.43.
2. Increase the exponent by five to preserve the value: 3.43×10^{1}.
3. Check: $343000 \times 10^{-4} = 34.3$, and $3.43 \times 10^{1} = 34.3$. §4.1

Trap: moving the radix point changes the exponent in the opposite direction if the represented value is to stay the same.

Recall check · answer without looking

1. Which field chiefly determines range?
2. Which field chiefly determines precision?
3. Why are floating-point values not evenly spaced over the number line?

Answers: see "4.1 The floating-point model" under Answers at the end.

4.2 Binary fractions

Binary fractional place values explain both conversion and approximation.

The first bit after the binary point has weight 2^{-1}.

Each following bit has half the previous weight.

Convert binary fractions to decimal

Binary fractional place values

$$0.b_1b_2...b_k = b_1 2^{-1} + b_2 2^{-2} + ... + b_k 2^{-k}$$

Each b_i is 0 or 1. A leading integer part is evaluated with non-negative powers of 2.

Worked conversion · binary fraction to decimal

Convert 0.01101_2 to decimal.

1. Use the five place values: 1/2, 1/4, 1/8, 1/16, and 1/32.
2. Keep the places where the bit is 1: 1/4 + 1/8 + 1/32.
3. Add: 0.25+0.125+0.03125=0.40625. §4.2

Convert decimal fractions to binary

Repeatedly multiply the fractional part by 2.

At each step, the integer part is the next binary digit, and the new fractional part is carried into the next step.

Worked conversion · terminating binary fraction

Convert 0.6875 to binary.

1. $0.6875 \times 2 = 1.375$, so the first bit is 1; carry 0.375.
2. $0.375 \times 2 = 0.75$, so the next bit is 0; carry 0.75.
3. $0.75 \times 2 = 1.5$, so the next bit is 1; carry 0.5.
4. $0.5 \times 2 = 1.0$, so the next bit is 1; the fraction is now zero.
5. Therefore $0.6875 = 0.1011_2$. Check: $1/2 + 1/8 + 1/16 = 0.6875$. §4.2

Some fractions, such as 0.1 in decimal, repeat in binary.

A finite significand stores only an approximation of a repeating expansion.

Trap: a decimal fraction that looks short is not necessarily exact in binary.

Recall check · answer without looking

1. What is the weight of the fourth binary digit after the point?
2. What is 0.101_2 in decimal?
3. Why can a computer store 0.1 only approximately in binary floating point?

Answers: see "4.2 Binary fractions" under Answers at the end.

4.3 Floating-point arithmetic and error

The exponent and significand must be managed during each operation.

After the exact operation, the result may need normalisation and rounding to fit the available precision.

Multiplication

Multiply normalised values

$$(\mathrm{M}_1 \times \beta^{\mathrm{E}_1})(\mathrm{M}_2 \times \beta^{\mathrm{E}_2}) = (\mathrm{M}_1\mathrm{M}_2) \times \beta^{\mathrm{E}_1+\mathrm{E}_2}$$

Multiply the significands and add the exponents. Then normalise and round if needed.

Worked multiplication · normalise the result

Calculate $(2.6\times10^{3})(5.4\times10^{-2})$.

1. Multiply significands: $2.6\times5.4=14.04$.
2. Add exponents: $3+(-2)=1$, giving 14.04×10^{1}.
3. Normalise: $1.404\times10^{2}=140.4$. §4.3

Addition

Before adding, express both numbers with the same exponent.

Shift the significand of the value with the smaller exponent to the right by the exponent difference.

This alignment can discard low-order digits before the addition is even performed.

Worked addition · align exponents

Use two significant decimal digits to add 4.5×10^{3} and 6.7×10^{2}.

1. Rewrite the second value at exponent 3: $6.7\times10^{2}=0.67\times10^{3}$.
2. Add significands: $(4.5+0.67)\times10^{3}=5.17\times10^{3}$.
3. Round to two significant digits: 5.2×10^{3}. The exact sum is 5170, while this two-digit result is 5200. §4.3

Rounding, overflow, and comparison

Truncation drops extra digits and tends to bias results in one direction.

Rounding chooses a nearby representable value and usually gives a better approximation.

Exponent overflow occurs when the result is too large for the format.

Exponent underflow occurs when the result is too small for the normalised range.

The slides describe infinity or an exception for overflow, and zero or an exception for underflow in their simplified model.

IEEE formats also provide subnormal numbers near zero.

For approximate comparisons, choose a tolerance ε that fits the scale and precision of the problem.

Absolute tolerance example

$$\text{Treat a as close to b when } |a - b| < \varepsilon$$

$$\text{For } b = 1 \text{ and } \varepsilon = 0.000005 : \; 0.999995 < a < 1.000005$$

A fixed absolute tolerance may be unsuitable when magnitudes vary greatly. The slides also introduce relative closeness.

Sign check: estimate the order of magnitude before accepting a rounded result.

Recall check · answer without looking

1. What do you do to the exponents when multiplying?
2. Which operand's significand is shifted to align an addition?
3. How do overflow and underflow differ?
4. Why can direct equality comparison be unreliable for computed floating-point values?

Answers: see "4.3 Floating-point arithmetic and error" under Answers at the end.

4.4 IEEE 754 binary formats

A shared format defines how floating-point fields are stored and how arithmetic behaves.

The slides introduce IEEE 754 as a widely adopted standard for predictable binary floating-point arithmetic.

Single and double precision

Format	Total	Sign	Exponent	Fraction field	Significand precision	Bias
Single	32 bits	1 bit	8 bits	23 bits	24 bits for normal values	127
Double	64 bits	1 bit	11 bits	52 bits	53 bits for normal values	1023

For a normalised binary value, the leading significand bit is 1 and is omitted from the stored fraction field.

The exponent field stores a biased exponent rather than a two's-complement exponent.

Normalised value rule

$$\text{value} = (-1)^{S} \times 1.F \times 2^{E-\text{bias}}$$

$$\text{Single bias} = 127;\ \text{double bias} = 1023$$

S is the sign bit, E is the unsigned exponent field, and F is the stored fraction interpreted after the binary point. This rule is for normalised values.

Convert decimal to single precision

Worked conversion · 42.6875

1. Convert to binary: $42.6875 = 101010.1011_2$.
2. Normalise: 1.010101011×2^5, so the sign bit is 0.
3. Store the fraction after the hidden leading 1: 01010101100000000000000.
4. Bias the exponent: $5 + 127 = 132 = 10000100_2$.
5. The fields are 0 | 10000100 | 01010101100000000000000, or hexadecimal 422A C000. §4.4

Decode single precision

Worked decoding · BEC0 0000

1. The sign bit is 1, so the value is negative.
2. The exponent field is 01111101_2=125; subtract the bias: 125−127=−2.
3. Restore the hidden bit: $1.1000\ldots_2$=1.5.
4. Scale: 1.5×2^{-2}=0.375. Apply the sign: −0.375. §4.4

Trap: restore the hidden leading 1 when decoding a normalised value, but do not apply that rule to a subnormal value.

Recall check · answer without looking

1. How many bits are stored in the single-precision fraction field?
2. What is the single-precision exponent bias?
3. Why can a normalised value have 24 bits of significand precision while storing only 23 fraction bits?

Answers: see "4.4 IEEE 754 binary formats" under Answers at the end.

4.5 Zero, subnormal numbers, infinity, and NaN

Reserved exponent patterns extend useful behaviour beyond normalised values.

In single precision, exponent field zero and exponent field all ones have special interpretations.

Exponent field	Fraction field	Meaning
All zeroes	All zeroes	Positive or negative zero
All zeroes	Non-zero	Subnormal number
Neither all zeroes nor all ones	Any pattern	Normalised number
All ones	All zeroes	Positive or negative infinity
All ones	Non-zero	NaN, or not-a-number

For a subnormal single-precision number, the leading bit is 0 and the effective exponent is -126.

Subnormal values provide a gradual step from the smallest normalised values toward zero.

Infinity represents overflow results and some divisions by zero.

NaN represents an operation without a real-number result, such as $0/0$ or infinity$-$infinity.

Operations on infinity and NaN follow format rules, not ordinary real-number algebra in every case.

Special arithmetic examples

Operation	Result
N ÷ ±∞	0
± non-zero ÷ 0	±∞
+∞ + +∞	+∞
±∞ × ±∞	±∞
±0 ÷ ±0	NaN
+∞ − +∞	NaN
±∞ ÷ ±∞	NaN
±∞ × 0	NaN

The sign of the infinity in signed cases depends on the operand signs. Slides note that an invalid operation may return a quiet NaN or raise a signalling-NaN exception.

Worked classification · single precision fields

Classify sign 0, exponent 11111111, fraction 000…000.

1. The exponent field is all ones.
2. The fraction field is all zeroes.
3. This encodes positive infinity. §4.5

Trap: an all-ones exponent does not always mean infinity; inspect the fraction field to distinguish infinity from NaN.

Recall check · answer without looking

1. How do you distinguish infinity from NaN?
2. What is the main purpose of subnormal numbers?
3. What does zero exponent and zero fraction encode?

Answers: see "4.5 Zero, subnormal numbers, infinity, and NaN" under Answers at the end.

Formula map

Idea	Relation	Use
Floating-point model	$x = s \times M \times \beta^E$	Separate sign, significand, and scale.
Binary fraction	$0.b_1 b_2 \ldots = \Sigma\, b_i 2^{-i}$	Convert binary fractions to decimal.
Multiplication	$(M_1\beta^{E_1})(M_2\beta^{E_2}) = M_1 M_2 \beta^{E_1+E_2}$	Multiply significands and add exponents.
Single normalised value	$(-1)^S \times 1.F \times 2^{E-127}$	Decode a normalised single-precision pattern.
Double normalised value	$(-1)^S \times 1.F \times 2^{E-1023}$	Decode a normalised double-precision pattern.
Subnormal single value	$(-1)^S \times 0.F \times 2^{-126}$	Decode a subnormal single-precision pattern.
Absolute closeness	$\lvert a-b\rvert<\varepsilon$	Compare computed values with a chosen tolerance.

Check every answer

State the format and whether the value is normalised

Track the exponent when shifting the radix point

Check sign, scale, range, and final rounding

These are representation calculations, so verify the bit fields and magnitude rather than physical units.

Derive it cold

Cover the explanations and rebuild each result on blank paper.

1. **Binary fractions:** derive each bit's place value as a negative power of 2. §4.2
2. **Decimal fraction conversion:** use repeated multiplication by 2 and track the fractional remainder. §4.2
3. **Multiplication:** use exponent laws, then normalise. §4.3
4. **Addition:** align exponents, combine significands, and round. §4.3
5. **Single-precision decoding:** recover the sign, unbiased exponent, and hidden bit. §4.4
6. **Special values:** derive the classification from exponent and fraction fields. §4.5

Explain it

Answer each why or when question aloud before opening its model response.

1. Why does increasing the exponent change range more than precision?
2. Why can two nearby real values have different spacing when stored at different magnitudes?
3. Why does 0.1 decimal not have a finite binary fraction?
4. Why must exponents be aligned before adding two floating-point values?
5. Why can rounding during addition lose a small addend?
6. Why is the leading 1 omitted for normalised binary significands?
7. When would a subnormal number be used?
8. Why should an approximate comparison use a tolerance chosen for the problem?

Answers: see "Explain it" under Answers at the end.

Source check

The rendered slide images were checked for these range statements.

- **Slide 40 · approximate single-precision range endpoints are reversed:** it prints 10^{38} to 10^{-38} for the positive range. The increasing-magnitude order is approximately 10^{-38} to 10^{38}.
- **Slide 42 · approximate double-precision range endpoints are reversed:** it prints 10^{308} to 10^{-308} for the positive range. The increasing-magnitude order is approximately 10^{-308} to 10^{308}.

These are rough decimal magnitudes for normalised values, not exact endpoint values.

The later slides give the exact normalised exponent range for single precision.

The double-precision range follows from its exponent field size and bias.

Mixed practice

Try all 24 problems before opening the worked answers.

The topics are interleaved so you must choose the method each time.

1. Convert 0.1011_2 to decimal. §4.2
2. How many bits of significand precision does normalised IEEE single precision provide? §4.4
3. Normalise 0.000101_2 as $M \times 2^E$ with $1 \leq M < 2$. §4.1
4. Convert 0.375 decimal to binary. §4.2
5. In a floating-point format, which field chiefly determines range? §4.1
6. Multiply $(3 \times 10^2)(2 \times 10^{-4})$ and give a normalised result. §4.3
7. Align 7.2×10^5 to exponent 7. §4.3
8. How many stored fraction bits does double precision have? §4.4
9. Classify single-precision fields S=1, E=all ones, F=0. §4.5
10. Convert 0.01101_2 to decimal. §4.2
11. Use two significant decimal digits to round $8.46 \times 10^3 + 2.1 \times 10^2$. §4.3
12. What is the single-precision exponent bias? §4.4
13. Convert decimal 0.625 to binary. §4.2
14. For normalised single precision with exponent field E=130, what is the unbiased exponent? §4.4
15. State one reason truncation can be less desirable than rounding. §4.3
16. Classify single-precision fields E=0 and F non-zero. §4.5
17. For 1.25×2^3, give the decimal value. §4.1
18. How many total bits are used in double precision? §4.4
19. Give an absolute-tolerance test for a value a to be close to b using ε. §4.3
20. Convert 1.101_2 to decimal. §4.2
21. Why is 1.25×2^3 an exact binary value? §4.2
22. What do $\pm$non-zero $\div 0$ and $\pm 0 \div \pm 0$ produce? §4.5
23. What happens to the exponent when multiplying two floating-point values? §4.3
24. When the exponent field is all ones and the fraction is zero, what is the value when S=0? §4.5

Answers: see "Mixed practice" under Answers at the end.

Exam-style problems

Allow about 20 minutes per problem.

Show the format, fields, and each conversion step.

1 · Encode and decode single precision

Encode 13.25 as IEEE single precision, then decode the resulting fields to verify the value.

1. Convert 13.25 to binary.
2. Normalise it and identify sign, exponent, and stored fraction.
3. Give the exponent field and verify by decoding.

Answers: see "Exam-style problems" under Answers at the end.

2 · Align, add, and assess precision

Using two significant decimal digits, calculate $3.4\times10^4 + 5.6\times10^2$.

1. Align both values to exponent 4.
2. Add the significands and round to two significant digits.
3. Compare the rounded result with the exact decimal sum and explain the source of the difference.

Answers: see "Exam-style problems" under Answers at the end.

Self-check

Tick a skill only in a later session, at least one day after studying, when you can demonstrate it without notes.

- ☐ Explain how significand and exponent affect precision and range. (3, 5)
- ☐ Normalise a floating-point value and preserve its magnitude. (3)
- ☐ Convert binary fractions to decimal. (1, 10, 20)
- ☐ Convert terminating decimal fractions to binary. (4, 13)
- ☐ Explain why some decimal fractions repeat in binary. (21)
- ☐ Multiply floating-point numbers and normalise. (6)
- ☐ Align exponents before adding. (7, 11, 24)
- ☐ Round results to a stated precision. (11, 15, 24)
- ☐ State an approximate comparison using tolerance. (19)
- ☐ State the single-precision field sizes and bias. (2, 12)
- ☐ Encode a decimal value in single precision. (exam 1)
- ☐ Decode a normalised single-precision value. (14, exam 1)
- ☐ Classify a subnormal pattern and explain its role. (16)
- ☐ Classify special-value fields and predict basic infinity/NaN arithmetic. (9, 22, 24)
- ☐ Rebuild the derivations from the list.
- ☐ Complete both exam-style problems with checks.

Answers

Start from memory

1. The exponent controls scale and therefore the range of magnitudes. §4.1
2. Its binary expansion repeats, so a finite significand must approximate it. §4.2
3. Align the exponents by shifting the significand of the value with the smaller exponent. §4.3
4. Thirty-two bits: one sign, eight exponent, and twenty-three stored fraction bits. §4.4
5. For normalised binary numbers the leading significand bit is always 1, so it can be omitted from storage and treated as implicit. §4.4

4.1 The floating-point model

1. The exponent field. §4.1
2. The significand field. §4.1
3. Each exponent scales the significand by a different power of the base, so the gap between representable values grows with magnitude. §4.1

4.2 Binary fractions

1. $2^{-4}=1/16$. §4.2
2. $1/2+1/8=0.625$. §4.2
3. The binary expansion repeats, but the significand has a finite number of bits and must be rounded or truncated. §4.2

4.3 Floating-point arithmetic and error

1. Add them. §4.3
2. Shift the significand belonging to the smaller exponent to the right. §4.3
3. Overflow exceeds the maximum magnitude; underflow falls below the smallest normalised magnitude. §4.3
4. Finite precision can make a computed result differ slightly from its exact mathematical value, so a suitable tolerance may be needed. §4.3

4.4 IEEE 754 binary formats

1. Twenty-three. §4.4
2. 127. §4.4
3. The leading binary 1 is implicit, giving 24 significant bits including the 23 stored fraction bits. §4.4

4.5 Zero, subnormal numbers, infinity, and NaN

1. Both use an all-ones exponent. A zero fraction means infinity; a non-zero fraction means NaN. §4.5
2. They represent values between zero and the smallest normalised magnitude, making the transition toward zero gradual. §4.5
3. Positive or negative zero, according to the sign bit. §4.5

Explain it

1. The exponent scales a significand by powers of the base, expanding which magnitudes can be represented. Precision depends chiefly on how many significand bits distinguish values at a given scale.
2. At a larger exponent, the same significand step is multiplied by a larger power of 2. Therefore adjacent representable values are farther apart.
3. A finite binary fraction has a denominator that is a power of 2 after reduction. One tenth has a factor of 5 in its denominator, so its binary expansion repeats.
4. Only values with the same exponent have significands describing the same scale. Alignment expresses both at a shared scale before combining them.
5. Alignment shifts the smaller significand to the right. If the exponent difference is large, its significant bits may fall beyond the available precision.
6. Every normalised binary number begins with 1 before the point, so the format can infer that bit and use its storage for another fraction bit.
7. A subnormal represents a non-zero value smaller than the minimum normalised magnitude, close to zero.
8. Absolute or relative error that is acceptable depends on the scale and purpose of the calculation. One fixed tolerance may be too strict for large values or too loose for small ones.

Mixed practice

1. $1/2+1/8+1/16=0.6875$. §4.2
2. 24 bits: the 23 stored fraction bits plus the implicit leading 1. §4.4
3. $1.01_2\times2^{-4}$. The point moves four places right to make the significand 1.01, so the exponent is -4. §4.1
4. $0.375=3/8=0.011_2$. §4.2
5. The exponent field. §4.1
6. Multiply significands: 6. Add exponents: -2. Result 6×10^{-2}, normalised as 6×10^{-2}. §4.3
7. $7.2\times10^5=0.072\times10^7$; shift the significand two places right as the exponent rises by two. §4.3
8. 52 stored fraction bits. §4.4
9. Negative infinity: all-ones exponent and zero fraction mean infinity; sign bit 1 makes it negative. §4.5
10. $1/4+1/8+1/32=0.40625$. §4.2
11. Align $2.1\times10^2=0.21\times10^3$. Sum 8.67×10^3; to two significant digits this is 8.7×10^3. §4.3
12. 127. §4.4
13. $0.625=1/2+1/8=0.101_2$. §4.2
14. $130-127=3$. §4.4
15. Truncation always discards in the same direction for positive values and can introduce a biased error. §4.3
16. A subnormal number. The exponent is zero and the fraction is non-zero. §4.5
17. $1.25\times8=10$. §4.1
18. 64 bits: one sign, eleven exponent, and 52 fraction bits. §4.4
19. For example, treat a as close to b when $|a-b|<\varepsilon$, with ε chosen for the expected scale and precision. §4.3
20. $1+1/2+1/8=1.625$. §4.2
21. $1.25=1.01_2$ is a finite binary fraction, and multiplying by 2^3 only shifts the binary point. §4.2
22. $\pm\infty$ and NaN, respectively. The first is division of a non-zero value by zero; zero divided by zero has no real result. §4.5
23. Add the exponents and multiply the significands, then normalise if needed. §4.3
24. Positive infinity. §4.5

Exam-style problems

1. $13.25=1101.01_2$, since $13=1101_2$ and $0.25=0.01_2$. §4.2
2. Normalise: 1.10101×2^3. The sign is 0. The stored fraction is 10101000000000000000000 after padding to 23 bits. §4.4
3. Biased exponent is $3+127=130=10000010_2$. The pattern is 0 | 10000010 | 10101000000000000000000. Decoding restores $1.10101_2\times2^3=13.25$. §4.4

Exam-style problems

1. $5.6\times10^{2}=0.056\times10^{4}$. §4.3
2. $(3.4+0.056)\times10^{4}=3.456\times10^{4}$, which rounds to 3.5×10^{4}. §4.3
3. The exact sum is 34560 and the rounded result is 35000. The smaller addend contributes digits beyond the two-digit precision, so rounding changes the stored result. §4.3

Personal study guide based on the supplied "04_floating_point_numbers.pdf" (50 slides; slide 2 is dated 22 April 2024). Written in my own words; no lecture pages or figures are reproduced. Not an official FAU publication; check course details against the current notes.